CHAPTER · WEEK 2

Linux for DevOps

The server is your workplace. Learn to live on the command line.

6 sessions · 17 tool families · one mental model

Instructor · Sagyam Thapa

The week at a glance

Each day builds on the last: orient → control access → transform text → run the system → connect it → automate it.

DAY 1

Living in the Filesystem

Navigate, create, copy, edit (vim/nano)

DAY 2

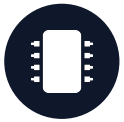
Users & Permissions

Accounts, groups, ownership, rwx & chmod

DAY 3

Text Processing

grep · sed · awk · cut · jq · yq



DAY 4

Administering the System

apt · processes · services · logs · cron

DAY 5

Networking & Remote

ip · dig · ssh · ufw · curl/wget

DAY 6

Scripting & Storage

Bash scripts · tar/zip · LVM

DAY 1

Living in the Filesystem

First, how Linux is actually built — then orient yourself, manipulate files, and edit text, all from the terminal.

the stack | distros | navigation | files & dirs | the FHS | vim · nano

the command line is the DevOps workplace

How Linux is actually built

Why you care: *"Linux" is just the kernel. Everything you call an OS is layers stacked on top — and DevOps lives in those layers.*

Applications
your API, nginx, scripts, containers

Shell & user space
bash / zsh + GNU coreutils: ls, grep, cp

Init system — systemd
PID 1: starts & supervises every service

Kernel (this is "Linux")
talks to hardware: CPU, memory, devices, syscalls

Hardware
CPU · RAM · disk · network card

BOOT ORDER

Firmware (UEFI/BIOS)
powers on, finds the boot disk

↓

Bootloader · GRUB
loads the kernel into memory

↓

Kernel
mounts root, starts PID 1

↓

systemd
brings up services + a login

What is virtualization?

Why you care: *every cloud server you'll rent — EC2, a Droplet, an Azure VM — is a virtual machine. This is the abstraction the whole cloud is built on.*

The idea

One physical server, sliced into several isolated virtual machines. Each VM is fooled into thinking it owns real hardware — its own CPU, RAM, disk, NIC.

The hypervisor does the slicing

A thin software layer sitting between the hardware and the VMs. It rations real CPU and RAM to each guest and keeps them walled off from each other.

Each VM runs a full OS

Its own kernel, its own boot sequence — a complete computer that just happens to be made of software.

THE ANALOGY · APARTMENT BUILDING

The plot of land
= one physical server

The building + plumbing
= the hypervisor that splits the space

Each apartment
= a VM: own kitchen, bath, locked door

The tenants
= isolated; one floods, others stay dry

One plot, many homes = *consolidation + isolation*

Hypervisors: Type 1 vs Type 2

Why you care: *Type 1 is what every cloud runs on; Type 2 is what's on your laptop. Knowing which is which tells you where the performance goes.*

Type 1 — bare-metal

Runs **directly on the hardware**. Fast, data-center grade. KVM, VMware ESXi, Hyper-V, Xen. Every EC2 instance lives here.

Type 2 — hosted

Runs **as an app on top of a normal OS**. Great for dev. VirtualBox, VMware Workstation. Slower — there's a whole OS underneath it.

Not the same as a container

A VM virtualizes **hardware** (full guest OS, GBs, boots in minutes). A container virtualizes the **OS** (shares the host kernel, MBs, starts in seconds). Containers often run inside VMs.

WHY DEVOPS CARES

Consolidation

one beefy server runs many workloads, not idle boxes

Isolation

one VM crashes or is breached, the neighbours survive

Snapshots + clones

freeze state, roll back, or clone a box in seconds

The cloud itself

AWS, Azure, GCP are virtualization farms you rent

What is a distribution ("distro")?

Why you care: *apt works on Ubuntu but not on Alpine — the distro decides your package manager, your libc, and your base image.*

A DISTRO BUNDLES

kernel	the Linux kernel (shared by all)
userland	coreutils + libc (GNU, or musl)
pkg manager	apt · dnf · apk · pacman
init	usually systemd
defaults	file layout, configs, release cadence

```
you@server

$ cat /etc/os-release
NAME="Ubuntu"
VERSION="24.04 LTS"
$ uname -srm
Linux 6.8.0-31 x86_64  # the kernel
```

Same kernel, different wrapping

Your Docker base image — **ubuntu, debian, alpine** — IS a distro choice. Families: **Debian/Ubuntu→apt · RHEL/Fedora→dnf · Alpine→apk · Arch→pacman**

Orient & inspect

Why you care: *You SSH into an unfamiliar server — first you need to know where you are and what's there.*

LOOK AROUND

`pwd`	print working directory
`ls -lah`	long list, all, human sizes
`tree -L 2`	directory tree, 2 levels deep
`date`	current date/time (logs, naming)
`df -h`	disk free per mount, human-readable

```
you@server: ~

$ df -h /
Filesystem  Size Used Avail Use%
/dev/sda1    40G  31G  9.2G  78%

$ ls -lah /var/log
-rw-r----- syslog  2.1M nginx
-rw-r----- syslog  118K auth.log

# 78% disk + huge logs = a clue
```

Create, copy & move

Why you care: *Setting up a deploy: you scaffold directories, drop in files, and copy a config into place.*

MAKE & MOVE

`touch f`	create empty file / update mtime
`mkdir -p`	make dirs, parents as needed
`cp -r a b`	copy recursively (dirs)
`cp -a`	copy + preserve perms/owner/time
`mv a b`	move or rename (same command)
`rm -r`	remove recursively — no undo

```
you@server: ~

$ mkdir -p app/{conf,logs}
$ touch app/conf/app.env
$ cp -a /tmp/nginx.conf \
      app/conf/
$ tree app
app
├── conf
│   ├── app.env
│   └── nginx.conf
└── logs
```

Where things live: the filesystem hierarchy

Why you care: *Logs, configs, and binaries live in predictable places — knowing the map is half of troubleshooting.*

`/etc`
system & service config — what you edit

`/var/log`
log files — first stop when it breaks

`/var/lib`
service state & data (db files, etc.)

`/home`
per-user home directories

`/root`
the root user's home

`/tmp`
scratch space — cleared on reboot

`/usr/bin · /bin`
installed command binaries

`/usr/local · /opt`
third-party & locally-installed software

`/boot`
kernel + bootloader files

`/dev`
device files (disks, terminals)

`/proc · /sys`
virtual: live kernel & process state

`/mnt · /media`
mount points for extra/removable disks

Everything is a file in Linux — including devices (/dev) and live kernel state (/proc).

The whole tree at a glance

Why you care: *One picture of the root filesystem — so the paths from the last slide click into place.*

```
you@server:~$  tree -L 1 /

/
├── bin   → /usr/bin   commands
├── boot       kernel + GRUB
├── dev        devices (disks, tty)
├── etc        config you edit
├── home       user homes
├── opt        3rd-party software
├── proc       live kernel state
├── root       root's home
├── tmp        scratch (wiped on boot)
├── usr        programs + libraries
└── var        logs, data, spool
```

Reading it

Everything hangs off **/** ("root") — there are no drive letters. **tree -L 1 /** shows the top level; raise **-L** to go deeper.

The two in orange

/etc and **/var** are where you'll spend nearly all your time — configs in one, logs and data in the other.

vim — it's on every server

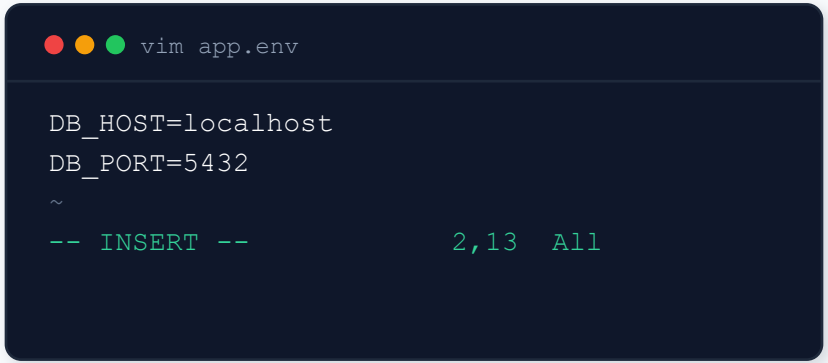
Why you care: *There's no GUI on a production box. vim is installed everywhere, so you must be able to escape it.*

SURVIVAL KIT

`i`	INSERT mode — start typing
`Esc`	back to NORMAL mode
`:w`	write (save)
`:q  :q!`	quit / quit without saving
`:wq  ZZ`	save and quit
`/text`	search; n = next match
`dd  yy  p`	delete / yank / paste a line

The one idea that unlocks vim

vim is modal. You're either typing text (INSERT) or issuing commands (NORMAL). Stuck? Press Esc, then :q! to bail.



Search with your fingers, not your eyes

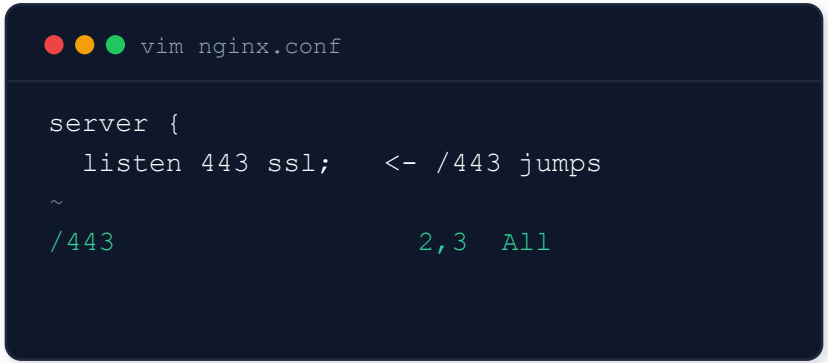
Why you care: *Beginners hunt with their eyes and arrow keys — vim lets you name a target and jump straight to it.*

JUMP, DON'T SCAN

`/text  ?text`	search forward / back
`n  N`	next / previous match
`*  #`	search word under cursor
`f{c}  ;`	jump to char; ; repeats
`t{c}  ,`	till before char; , back
`w  b  e`	word: next / back / end
`:42  gg  G`	go to line / top / bottom

The shift: declare, don't drive

Don't arrow over and hunt with your eyes. Name the target — a word, a character, a line — and let vim jump straight to it.



nano — the friendly fallback

Why you care: *Quick one-line config edit and you don't want to think? nano shows its shortcuts on screen.*

NANO KEYS (^ = Ctrl)

`nano f`	open/create file f
`^O`	write Out (save)
`^X`	exit
`^W`	where is (search)
`^K  ^U`	cut line / paste (uncut)

Which editor, when?

nano — quick edits, beginners, when shortcuts-on-screen help.

vim — the only one guaranteed present; worth real fluency over time.

```
nano app.env

DB_HOST=localhost

^G Help    ^O Write Out    ^W Where
^X Exit    ^K Cut          ^U Paste
```

DAY 2

Users, Groups & Permissions

Who is allowed to do what — the foundation of every security decision on the box.

users | groups | ownership | chmod | chown | special bits

the command line is the DevOps workplace

Why a server has many users

Why you care: *nginx, postgres, your deploy bot — each runs as its own user so a compromise stays contained.*

root (uid 0)

The superuser. Can do anything — which is exactly why services should NOT run as root.

Service users

nginx, postgres, redis… no login shell, minimal rights. Least privilege in action.

Human users

You and your team. Get root only when needed, via sudo — never log in as root.

```
you@server

$ id            # who am I, what groups am I in?
uid=1000(sagyam) gid=1000(sagyam) groups=1000(sagyam),27(sudo),999(docker)
$ ps -u postgres    # postgres runs as its own user, not root
```

Managing users & groups

Why you care: *Onboarding a teammate or creating a service account is a routine, scriptable task.*

ACCOUNTS

`adduser bob`	interactive: user + home + group
`useradd -r`	low-level: -r = system acct
`passwd bob`	set/change a password
`usermod -aG`	ADD to a group (-a is vital!)
`groupadd dev`	create a group
`deluser bob`	remove a user

root@server

```
# add deploy user, no password login
$ useradd -m -s /bin/bash \
        -G docker deploy

# grant an existing user sudo
$ usermod -aG sudo sagyam

# verify
$ groups deploy
deploy : deploy docker
```

Reading permissions: the rwx model

Why you care: *Every 'permission denied' is solved by reading one line of ls -l correctly.*

```
-rwxr-xr--  1 deploy www-data  deploy.sh
type   user group other     owner   group
```

r = 4
read — view file / list directory

w = 2
write — modify file / add-remove in dir

x = 1
execute — run file / enter (cd) a dir

The trap: on a directory, **x** doesn't mean "run" — it means "enter". Without it you can't cd in, even with r.

chmod — changing permissions

Why you care: *Your deploy script isn't running, or a key is world-readable — chmod is the fix.*

TWO NOTATIONS

`chmod +x f`	symbolic: make executable
`chmod u+w`	owner gains write
`chmod go-r`	group+other lose read
`chmod 755`	numeric: rwx r-x r-x
`chmod 644`	numeric: rw- r-- r--
`chmod -R`	recurse into a directory

Add the numbers: r4 + w2 + x1

`7=rwx` `6=rw-` `5=r-x` `4=r--`

755 = owner full, everyone else read+enter. 644 = owner edits, others read.

you@server

```
$ chmod +x deploy.sh
$ ./deploy.sh    # now it runs

# SSH refuses keys that others can read
$ chmod 600 ~/.ssh/id_ed25519
```

Ownership & the special bits

Why you care: *Your web server can't write uploads — usually it's the wrong owner, not the wrong mode.*

OWNERSHIP

`chown u f`	change owner
`chown u:g f`	change owner AND group
`chown -R`	recurse (whole tree)
`chgrp g f`	change group only

Special bits (advanced)

setuid (4) run as file's owner (e.g. `passwd`)

setgid (2) new files inherit the dir's group

sticky (1) only owner can delete (e.g. `/tmp`)

root@server

```
# hand the web root to the nginx user
$ chown -R www-data:www-data /var/www/app
$ ls -ld /var/www/app
drwxr-xr-x  www-data www-data  /var/www/app
```

DAY 3

Text Processing & Filtering

Logs, configs, API responses — it's all text. The engineer who can slice text wins.

pipes | grep | cut | sed | awk | jq | yq

the command line is the DevOps workplace

The Unix philosophy: small tools, piped

Why you care: *One giant command rarely exists — you build the answer by chaining simple tools.*

```
cat access.log | grep ' 500 ' | awk '{print $1}' | sort | uniq -c
```

"count how many distinct IPs hit my server with a 500 error" — assembled from four small tools.

stdin / stdout

Tools read from input, write to output. The pipe | wires one's output to the next's input.

stderr

Errors flow on a separate channel (2) so they don't pollute your data. Redirect with 2>.

redirection

> writes to a file, >> appends, < reads from a file. Combine with pipes freely.

grep — find lines that match

Why you care: *Something failed at 14:32. grep is how you find the needle in a 2-million-line log.*

grep [opts] pattern file

`-i`	case-insensitive
`-r`	recurse through directories
`-n`	show line numbers
`-v`	invert: lines that DON'T match
`-c`	count matching lines
`-E`	extended regex (or use egrep)
`-A2 -B2`	show lines After / Before

you@server

```
# all errors, case-insensitive
$ grep -i error app.log

# errors with 2 lines of context
$ grep -n -A2 ERROR app.log
412:ERROR db timeout
413-  retrying in 5s

# pipe: 500s, excluding health checks
$ grep ' 500 ' a.log | grep -v /health
```

cut, sort, uniq — columns & counting

Why you care: *A CSV or colon-delimited file, and you need just one field, deduped and counted.*

FIELD TOOLS

`cut -d: -f1`	field 1, ':' delimiter
`cut -f2,5`	fields 2 and 5 (tab default)
`sort`	sort lines; -n numeric, -r reverse
`uniq -c`	collapse dupes + count (needs sort)
`wc -l`	count lines
`head / tail`	first / last N; tail -f follows

you@server

```
# all usernames on the box
$ cut -d: -f1 /etc/passwd
root  daemon  nginx  ...

# top 3 IPs by request count
$ cut -d' ' -f1 access.log \
   | sort | uniq -c \
   | sort -rn | head -3
  812 203.0.113.7
  344 198.51.100.2
```

sed — edit a stream

Why you care: *Flip a config flag across 50 files in CI, without opening an editor.*

sed 'script' file

`s/a/b/`	substitute first a → b per line
`s/a/b/g`	substitute ALL on the line
`s/a/b/gi`	...case-insensitive too
`-i`	edit file IN PLACE (careful)
`-i.bak`	...keep a .bak backup
`/re/d`	delete matching lines
`-n '5,8p'`	print only lines 5–8

you@server

```
# preview (no -i): debug -> info
$ sed 's/debug/info/' app.conf

# apply in place, keep a backup
$ sed -i.bak \
    's/^ENV=.*/ENV=prod/' app.env

# strip blank lines and comments
$ sed '/^#/d; /^$/d' nginx.conf
```

awk — field-aware processing

Why you care: *You need column 4 summed, or a quick report from a log — awk is a tiny language for tables.*

awk 'pattern{action}'

`{print $1}`	first field of each line
`$NF`	the LAST field
`-F:`	set field separator to ':'
`$3>100`	pattern: rows where col 3 > 100
`{s+=$4}END`	accumulate; END runs at finish
`NR==1`	act only on line number 1

you@server

```
# sum bytes sent (field 10)
$ awk '{s+=$10} END{print s}' \
      access.log
48213776

# status code + URL, only 5xx
$ awk '$9>=500 {print $9,$7}' \
      access.log
503 /api/checkout
500 /api/orders
```

jq — query & transform JSON

Why you care: *Every API and most cloud CLIs return JSON. jq turns that wall of braces into the one value you want.*

curl ... | jq 'filter'

`.`	the whole input (pretty-print)
`.name`	value of key 'name'
`.items[]`	iterate array elements
`.items[].id`	id of every item
`select(.x>3)`	keep matching elements
`-r`	raw output (no quotes) — for scripts
`map(.) | length`	transform / count

you@server

```
# names of all running pods
$ kubectl get pods -o json \
    | jq -r '.items[].metadata.name'
web-7d9f   api-55c8   ...

# only unhealthy items, as a table
$ cat health.json | jq -r \
  '.svc[]|select(.up==false)|.name'
payments
```

yq — jq, but for YAML

Why you care: *Kubernetes manifests, CI pipelines, docker-compose, Ansible — DevOps config is YAML.*

yq 'expr' file.yaml

`.spec.replicas`	read a nested value
`-i '...=3'`	edit YAML in place
`e '.a.b'`	evaluate an expression
`-o=json`	convert YAML → JSON
`ea 'sel...'`	across multiple docs (---)

Heads-up: use mikefarah's Go `yq` (the syntax here). The pip/Python `yq` is different.

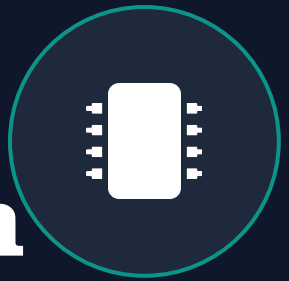
you@server

```
# read replica count from a manifest
$ yq '.spec.replicas' deploy.yaml
3

# bump it to 5, in place
$ yq -i '.spec.replicas=5' \
    deploy.yaml
# convert to JSON for jq
$ yq -o=json deploy.yaml | jq .
```

DAY 4

Administering the System



Install software, watch processes, run services, read logs, and schedule work.

apt | ps / btop | systemctl | services | journalctl | cron | timers

apt — installing software (Ubuntu/Debian)

Why you care: *Step one on any fresh box: update the index and install the tools you need.*

apt <subcommand>	
`apt update`	refresh package index (do first)
`apt upgrade`	install available updates
`apt install`	install a package
`apt remove`	remove (purge also drops config)
`apt search`	find a package by keyword
`apt show`	details about a package
`apt list --installed`	what's installed

root@server

```
# the canonical first two lines
$ apt update && apt -y upgrade

$ apt install -y nginx jq curl
Setting up nginx (1.24)...

# why is this command missing?
$ apt search ^htop$
htop - interactive process viewer
```

Watching processes: ps & btop

Why you care: *CPU is pinned or memory's gone — you need to see what's running and kill the culprit.*

PROCESSES

`ps aux`	every process + CPU/MEM
`ps -ef`	every process + parent PID
`pgrep -f x`	find PIDs by name/cmdline
`kill PID`	send TERM (graceful stop)
`kill -9 PID`	send KILL (force; last resort)
`btop`	live dashboard (CPU/mem/net/proc)

```
you@server

# top memory hogs
$ ps aux --sort=-%mem | head -4
USER  PID %CPU %MEM  COMMAND
app  9241  2.0 41.3  java
app  7180  0.4  6.1  node

# find and stop a runaway
$ pgrep -f stress
9412
$ kill 9412
```

systemd: the thing that runs everything

Why you care: *Why does nginx restart on boot and come back after a crash? Because systemd manages it.*

It's the init system

PID 1. Starts everything at boot and supervises it after.

Everything is a unit

.service (daemons), .timer (schedules), .socket, .mount…

It restarts & orders

Auto-restart on failure, dependency ordering, resource limits.

Service lifecycle you'll drive:

enabled (boot) → started (now) → running → reload / restart → stopped

"enabled" = starts at boot. "started/active" = running right now. They are independent — know which one you mean.

systemctl — controlling services

Why you care: *Deploy a new build and you need to restart the service and confirm it actually came back.*

systemctl <verb> <unit>

`status nginx`	state + recent log lines
`start / stop`	run / halt now
`restart`	stop then start
`reload`	re-read config, no downtime
`enable --now`	start now AND on boot
`disable`	don't start on boot
`is-active`	scripts: running? (exit code)

you@server

```
$ systemctl restart nginx
$ systemctl status nginx
● nginx.service - high perf web
  Active: active (running)
  Main PID: 9821 (nginx)

# survive reboots too
$ systemctl enable --now nginx
$ systemctl is-active nginx
active
```

Creating your own service

Why you care: *You wrote an app or a worker — wrap it in a unit so systemd keeps it alive.*

/etc/systemd/system/myapp.service

```
[Unit]
Description=My API
After=network.target

[Service]
User=app
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/run
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

THEN ACTIVATE IT

`daemon-reload`	re-read unit files
`enable --now`	start + run on boot
`status myapp`	confirm it's active

Three sections

`[Unit]` what & ordering `[Service]` how to run (incl. `Restart`) `[Install]` how it's enabled

journalctl — reading the logs

Why you care: *A service is failing on boot. journalctl is how you ask systemd exactly why.*

journalctl [opts]

`-u nginx`	logs for one unit
`-f`	follow live (like tail -f)
`-e`	jump to the end
`--since '1h ago'`	time-filter
`-p err`	priority: errors and worse
`-b`	this boot only; -b -1 = previous
`-n 50`	last 50 lines

```
you@server

# why did myapp fail? errors, this boot
$ journalctl -u myapp -p err -b
myapp[9412]: FATAL: port 8080 in use

# watch a deploy live
$ journalctl -u myapp -f
myapp[9650]: listening on :8080

# everything since 9am
$ journalctl --since 09:00
```

cron — classic scheduled jobs

Why you care: *Nightly backups, hourly cleanup, a health check every 5 minutes — cron has run these for decades.*

crontab -e

`crontab -e`	edit YOUR jobs
`crontab -l`	list your jobs
`* * * * *`	min hr dom mon dow → cmd
`*/5 * * * *`	every 5 minutes
`0 2 * * *`	every day at 02:00
`@reboot`	once, at boot

The five fields

`min` (0-59) `hr` (0-23) `dom` (1-31)

`mon` (1-12) `dow` (0-6, Sun=0)

crontab -e

```
# backup db nightly at 2am
0 2 * * * /opt/backup.sh

# health check every 5 min
*/5 * * * * /opt/check.sh
>> /var/log/check.log 2>&1
```

systemd timers — the modern scheduler

Why you care: *Same job as cron, but with logging, dependencies, and catch-up for missed runs.*

backup.timer + backup.service

```
# backup.timer
[Unit]
Description=Nightly backup
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
[Install]
WantedBy=timers.target

# backup.service runs the work
[Service]
ExecStart=/opt/backup.sh
```

MANAGE & INSPECT

`enable --now`	activate backup.timer
`list-timers`	next + last run of all timers
`OnCalendar=`	when (rich calendar syntax)
	run missed jobs after

Why timers over cron?

Logs go to journald, jobs can depend on other units, Persistent=true catches up missed runs, and OnCalendar is far more readable.

DAY 5

Networking, Remote & Transfer

Inspect the network, reach other machines safely, lock the door, and move files.

ip | ping / dig | ss | ssh | ufw | curl / wget

the command line is the DevOps workplace

ip — addresses, routes, links

Why you care: *What's my IP? Why can't this box reach the gateway? The ip command answers both.*

ip <object>

`ip a`	addresses on every interface
`ip r`	routing table (default gateway)
`ip link`	interfaces up/down + MACs
`ip -br a`	brief, scannable address view
`hostname -I`	just my IP address(es)

Modern note: `ip` replaces the old `ifconfig` / `route`. You'll still see those in old guides.

you@server

```
$ ip -br a
lo      UNKNOWN  127.0.0.1/8
eth0    UP       10.0.1.23/24

$ ip r
default via 10.0.1.1 dev eth0
10.0.1.0/24 dev eth0
```

Reachability, DNS & open ports

Why you care: *"The site's down" — is it the network, DNS, or is nothing even listening on the port?*

DIAGNOSE

`ping host`	is it reachable? (ICMP)
`dig +short x`	resolve name → IP, terse
`dig MX x`	specific record type
`ss -tulpn`	listening TCP/UDP ports + PID
`netstat -tulpn`	older equivalent of ss
`curl -I url`	headers only — is HTTP alive?

```
you@server

$ dig +short api.example.com
203.0.113.42

$ ping -c 2 203.0.113.42
64 bytes ... time=11.4 ms

# is anything listening on 8080?
$ ss -tulpn | grep 8080
tcp LISTEN 0 511 *:8080 users:
  (("myapp",pid=9650))
```

ssh — your way into every server

Why you care: *You will never sit at a server. SSH is the encrypted door to all of them.*

SSH TOOLKIT

`ssh u@host`	log in
`ssh -i key`	use a specific private key
`ssh-keygen`	make a keypair (-t ed25519)
`ssh-copy-id`	install your key on a host
`scp f u@h:/p`	copy files over SSH
`ssh u@h 'cmd'`	run one command remotely

~/.ssh/config

```
Host prod
  HostName 203.0.113.42
  User deploy
  IdentityFile ~/.ssh/id_ed25519
# now just:  ssh prod
```

Keys, not passwords

Generate a keypair, put the PUBLIC key on the server (ssh-copy-id), keep the PRIVATE key secret (chmod 600). No password to phish or brute-force.

ufw — the simple firewall

Why you care: *A public server with an open database port is a breach waiting to happen. ufw closes the doors.*

ufw <rule>

`ufw status numbered`	current rules, with index
`ufw allow 22/tcp`	permit SSH
`ufw allow 80,443`	permit web
`ufw deny 3306`	block MySQL from outside
`ufw delete 3`	remove rule #3
`ufw enable`	turn the firewall on

Don't lock yourself out! **ALWAYS `ufw allow 22/tcp` BEFORE `ufw enable`** — or your SSH session dies with no way back.

root@server

```
$ ufw allow 22/tcp
$ ufw allow 80,443/tcp
$ ufw enable
$ ufw status numbered
[1] 22/tcp    ALLOW IN   Anywhere
[2] 80,443    ALLOW IN   Anywhere
```

curl & wget — pulling from the web

Why you care: *Download a release, hit an API, run a health check — these two fetch anything over HTTP.*

FETCH & CALL

`curl -O url`	save with its remote filename
`curl -L`	follow redirects
`curl -fsSL`	fail-quiet-silent-loc (scripts)
`curl -H -d -X`	header / body / method (APIs)
`wget -O f url`	download to file f
`wget -c`	resume a broken download

```
you@server

# POST JSON to an API
$ curl -X POST api/login \
   -H 'Content-Type: application/json' \
   -d '{"user":"sagyam"}'

# fetch + parse in one pipe
$ curl -s api/health | jq .status
"ok"
```

Security: `curl ... | bash` runs whatever the server sends, as you. Read scripts before piping them to a shell.

DAY 6

Scripting, Archiving & Storage

Turn everything you learned into repeatable scripts — then package and store the results.

bash | variables | conditionals | loops | tar / zip | LVM

the command line is the DevOps workplace

Bash scripting: the basics

Why you care: *You typed the same five commands three times today — put them in a script and never retype them.*

deploy.sh

```
#!/usr/bin/env bash
set -euo pipefail

# variables: NO spaces around =
APP=myapp
DIR="/opt/$APP"

# positional args + read
VERSION="$1"
echo "Deploying $APP $VERSION"
echo "into $DIR"
```

ESSENTIALS

`#!/.../bash`	shebang: which interpreter
`chmod +x`	make the script runnable
`VAR=value`	assign (no spaces!)
`"$VAR"`	always quote expansions
`$1 $@ $#`	args: first / all / count

`set -euo pipefail` = exit on error, on unset vars, and on any pipe failure. Use it in every script.

Command substitution & exit codes

Why you care: *Scripts need to capture output and react to whether a command succeeded.*

CAPTURE & CHECK

`$(cmd)`	capture a command's output
`VAR=$(date +%F)`	today's date into a var
`$?`	exit code of last command
`0 = success`	non-zero = failure
`a && b`	run b only if a succeeded
`a || b`	run b only if a failed

backup.sh

```
# timestamped filename
STAMP=$(date +%Y%m%d-%H%M)
FILE="backup-$STAMP.tar.gz"

# act on success / failure
if pg_dump app > db.sql; then
  echo "dump ok"
else
  echo "dump FAILED ($?)" >&2
  exit 1
fi
```

Conditionals: making decisions

Why you care: *A deploy script should check things first — does the dir exist? is the arg present?*

TESTS [[…]]

`-f path`	file exists?
`-d path`	directory exists?
`-z "$v"`	string is empty?
`-n "$v"`	string is non-empty?
`= !=`	string equal / not equal
`-eq -gt -lt`	numeric ==, >, <

deploy.sh

```
if [[ -z "$1" ]]; then
  echo "usage: deploy VERSION" >&2
  exit 1
fi

if [[ -d "$DIR" ]]; then
  echo "updating existing"
else
  mkdir -p "$DIR"
fi

# case for multiple branches:
case "$ENV" in prod) ... ;; esac
```

Loops: doing it to many things

Why you care: *Restart five services, process every .log file, retry until a host responds.*

LOOP FORMS

`for x in a b c`	iterate a list
`for f in *.log`	iterate matching files
`for i in {1..5}`	a numeric range
`while read line`	line-by-line from input
`until cond`	loop until it's true
`break  continue`	exit / skip an iteration

ops.sh

```
# restart a list of services
for svc in nginx myapp redis; do
  systemctl restart "$svc"
  echo "restarted $svc"
done

# compress every old log
for f in /var/log/*.log; do
  gzip "$f"
done
```

Putting it together: a health-check script

Why you care: *Everything from this week — args, conditionals, loops, exit codes — in one real tool.*

healthcheck.sh

```
#!/usr/bin/env bash                  # Day 6: shebang
set -euo pipefail

SERVICES=(nginx myapp redis)          # an array
FAIL=0

for svc in "${SERVICES[@]}"; do         # Day 6: loop
  if systemctl is-active --quiet "$svc"; then   # Day 4 + cond
    echo "OK   $svc"
  else
    echo "DOWN $svc" >&2                # Day 3: stderr
    FAIL=$((FAIL + 1))
  fi
done
[[ "$FAIL" -eq 0 ]] && echo "all healthy" || exit 1  # exit code
```

Compress & archive

Why you care: *Bundle a release, ship logs off the box, snapshot a directory before a risky change.*

tar / gzip / zip

`tar -czf a.tgz d`	Create gZipped archive of d
`tar -xzf a.tgz`	eXtract it
`tar -tzf a.tgz`	lisT contents (don't extract)
`gzip f / gunzip`	compress / decompress one file
`zip -r a.zip d`	zip a folder (cross-platform)
`unzip a.zip`	extract a zip

tar vs gzip: two jobs

tar bundles many files into one (archiving); gzip shrinks a file (compression). -z does both at once, which is why .tar.gz is everywhere.

you@server

```
# snapshot before editing
$ tar -czf etc-$(date +%F).tgz \
        /etc/nginx
$ tar -tzf etc-2026-06-20.tgz
etc/nginx/nginx.conf
etc/nginx/sites-enabled/...
```

LVM — flexible storage

Why you care: *A partition fills up. With raw partitions you're stuck; with LVM you just grow it — live.*

Logical Volumes (LV)

/data, /var/log — what you format & mount, resizable

Volume Group (VG)

one big flexible pool of storage

Physical Volumes (PV)

the actual disks: /dev/sdb, /dev/sdc …

Why bother?

Grow a volume by adding a disk — no reformat

Snapshot before a risky upgrade

Span one filesystem across many disks

LVM in practice: create & grow

Why you care: *Provision a new data volume, then expand it when it fills — without downtime.*

root@server — create

```
# 1. disk -> physical volume
$ pvcreate /dev/sdb
# 2. pool it into a volume group
$ vgcreate data /dev/sdb
# 3. carve a logical volume
$ lvcreate -L 20G -n vol1 data
# 4. format and mount
$ mkfs.ext4 /dev/data/vol1
$ mount /dev/data/vol1 /data
```

root@server — grow

```
# add a disk to the pool
$ vgextend data /dev/sdc
# grow LV + filesystem (-r)
$ lvextend -L +10G -r \
      /dev/data/vol1
# verify
$ df -h /data
/dev/.../vol1   30G ... /data
```

pvs vgs lvs	inspect each layer

The week in one idea

Small tools. Plain text. Piped together. Saved into scripts. That's all of Linux ops — and all of DevOps automation.

When stuck, ask the box

man cmd · cmd --help · tldr cmd — the answer is already on the machine.

Read before you run

Preview with -t / dry runs; never pipe an unknown script straight to bash.

Everything chains

Day 3's pipes + Day 6's scripts + Day 4's timers = monitoring you built yourself.

Next: a hands-on scenario — you'll fix a broken server using only this week's tools.